The Pad-Safe SVD:
Exact Singular Value Decomposition of Zero-Padded Matrices

Companion notes for the T3Toolbox uniform layer
(`t3toolbox.backend.linalg.pad_safe_svd`; design record `dev/review_2026-08-22/repros/S1b/`)

August 2026

Abstract

Batched GPU/`jit` pipelines store matrices of varying sizes zero-padded to a common static shape, with boolean masks recording which rows and columns are real. A black-box SVD of such a padded matrix is *wrong* whenever the real block is numerically rank-deficient: the $\sigma \approx 0$ left singular vectors are an arbitrary orthonormal basis of a degenerate subspace that *contains the pad coordinates*, so they generically place mass on padded rows — and a downstream mask then erases them, silently losing a basis direction. In T3Toolbox this destroyed the uniform-layer tangent frame exactly at the rank-continuation warm start, the point the library's deflation strategy is built around. We surveyed the literature and found no batched, tolerance-free treatment: every rank-adaptive method either builds its new directions explicitly (outside the SVD) or pads and never asks the padded slots for orthonormality. These notes define the problem and the output contract, describe the shipped algorithm — *sketch–project* ("Method D"): a fixed orthonormal sketch, a QR with pads permuted to the trailing pivots, one small augmented SVD, and a right factor rebuilt from $A_{\text{pad}}^{\top} U = V\Sigma$ — which realizes the contract exactly, bitwise on the padding, with *no rank tolerance anywhere*, at cost $O(NM^2 + M^3)$ independent of the pad count, in one `jit` compilation for all mask patterns. An appendix records the alternatives we designed, measured, and rejected, including the two-sided augmentation and its equivalence with the generalized SVD, post-hoc completion, masked noise, and the measurements that settled the separation constant.

Contents

1 The problem **2**
- 1.1 Setting . . . 2
- 1.2 The failure mode . . . 2
- 1.3 Prior art . . . 3
- 1.4 The output contract . . . 3

2 The algorithm: sketch–project **4**
- 2.1 The permutation rule . . . 4
- 2.2 The four steps . . . 5
- 2.3 The separation constant . . . 5
- 2.4 Cost, batching, precision . . . 6
- 2.5 Integration into the T3 sweeps . . . 6

A Alternatives: designed, measured, rejected **7**
A.1 Method A: two-sided augmentation, and the GSVD view 7
A.2 Method B: post-hoc masked completion 7
A.3 Methods C, C′: masked noise, and Rayleigh–Ritz refinement 8
A.4 Round-tripping through the ragged layer 8
A.5 Shrinking the masks to the numerical rank 8
A.6 The symmetric contract needs no runtime transpose 8
A.7 The core is blind: why prevention beats repair 9
A.8 Summary of measurements 9

1 The problem

1.1 Setting

Let $A \in \mathbb{R}^{n\times m}$ be embedded in a fixed-shape padded matrix

$$A_{\text{pad}} \in \mathbb{R}^{N\times M}, \qquad N = n + P_r, \quad M = m + P_c,$$

where P_r row positions and P_c column positions — at *arbitrary, possibly interior* locations — are identically zero. Boolean masks of static length record the partition; in a batched setting the masks are runtime data and differ per batch element, while all array shapes are static (the requirement for a single `jit`/`vmap`/`scan` compilation).

In T3Toolbox this is the *uniform* representation of Tucker tensor trains: supercores padded to common edge widths with prefix rank masks, so that a whole stack of tensors of varying ranks flows through one compiled program [6]. Every orthogonalization step of the uniform sweeps — the Tucker down-orthogonalization, both TT sweeps, the TT up-orthogonalization, and the T3-SVD scan — is an SVD of a zero-padded unfolding, and the row index of an unfolding is often a Kronecker product of two padded edges, which is how *interior* padding arises even though each edge is padded only as a suffix.

1.2 The failure mode

If A has numerical rank $k < \min(n, m)$, the left null space of A_{pad} contains both the genuine null directions of A and the pad coordinates $\text{span}\{e_i : i \text{ padded}\}$. These are *one* degenerate eigenspace of $A_{\text{pad}}A_{\text{pad}}^\top$; a black-box SVD returns an arbitrary orthonormal basis of it, so the $\sigma \approx 0$ columns of U generically place mass on pad rows. Restricted to the real rows — which is what the masks do downstream — U is then non-orthonormal and possibly rank-deficient: **a silently lost basis direction**.

A useful minimal model (the "$k/z/p$ model"): the rows of an unfolding are k nonzero rows, z real-but-numerically-zero rows, and p padded rows. The first $r = \text{rank}(A)$ left singular vectors are determined by the data and automatically live in the k rows ($u = Av/\sigma$, and padded rows of Av are bitwise zero). The remaining vectors are *any* orthonormal basis of the complement, which spans $\mathbb{R}^{k+z}$ in the unpadded problem but $\mathbb{R}^{k+z} \oplus \mathbb{R}^p$ in the padded one. No factorization can keep them out of the p rows without being told the partition — an adversary can swap a z row for a p row without changing the matrix — so *the masks must enter the computation as data*. In floating point a roundoff-level σ is as unconstrained as an exact zero, so thresholding does not evade the problem, it only moves it.

Two facts sharpened the diagnosis in T3Toolbox. First, the truncating T3-SVD is *immune*: its completion columns are multiplied by $\sigma = 0$ on their way into the neighboring core, so the represented tensor is exact no matter where the completion lives (measured 60/60). The *frame* is not: its basis columns are the output. Second, the case that triggers the failure is precisely the library's selling point — the zero-padded `resize` warm start of rank continuation, which is structurally minimal but numerically deficient in both edge families. A padded restart has almost no gradient energy in the already-resolved eigenspace, which deflates the Newton–CG system; jitter would re-introduce it. Before the fix, that warm start lost tangent directions in 60/60 trials and the loss cascaded through the sweep.

1.3 Prior art

We looked for existing treatments (survey 2026-08-23; sources with links in the design record's `README`, under `repros/S1b/`). Nobody relies on the SVD's null-space completion to grow a basis:

- **TeNPy** `MPS.enlarge_chi`: zero-fills one bond; for the other, draws random rows, Gram–Schmidt-projects them against the existing ones, QR-orthonormalizes, and concatenates — explicit masked completion by hand, inside the real dimensions by construction.
- The **rank-adaptive BUG integrator** [3] augments $(K(t_1), U_0)$ and takes an orthonormal basis of its range by QR, noting the result is independent of the basis choice (the basis is gauge) and robust to small singular values.
- **Riemannian rank-adaptive TT completion** [4] never pads: the rank increase is a step along the *normal* part of the tangent cone, appended as new core columns — which is what a zero-padded warm start followed by one gradient step computes implicitly, and why the padded restart works.
- **DMRG subspace expansion** [5] appends a residual-derived term and lets the next SVD restore orthonormality; the literature contrasts this *structured* enrichment with White's density-matrix *noise*.
- **Vendors**: PyTorch's `linalg.svd` documentation states outright that trailing singular vectors "can be arbitrary bases of the corresponding subspaces"; cuSOLVER's batched kernels require equal sizes and say nothing about padded semantics; TensorLy's `pad_tt_rank` zero-pads "without changing the reconstruction" and is silent on orthogonality; JAX's ragged-data guidance is pad-and-mask, nothing SVD-specific.

None of these is batched, static-shape, and tolerance-free at once, which is what a `jit`-compiled sweep needs. That gap is what the algorithm below fills.

1.4 The output contract

Definition 1.1 (Pad-safe SVD)**.** Given $A_{\text{pad}} \in \mathbb{R}^{N \times M}$ with boolean masks marking n real rows and m real columns (padded entries bitwise zero), a *pad-safe SVD* returns $U \in \mathbb{R}^{N \times K}$, $s \in \mathbb{R}^{K}$, $V^\top \in \mathbb{R}^{K \times M}$ with $K = \min(N, M)$ such that, with $q = \min(n, m)$:

1. $A_{\text{pad}} = U\ \mathrm{diag}(s)\, V^\top$ — a genuine economy SVD of the padded matrix;

2. the first q triplets are a valid economy SVD of the unpadded real block A: its positive singular values, then real-supported null completions, with $U[:, :q]$ **bitwise** zero on padded rows and $V[:, :q]$ **bitwise** zero on padded columns (exact $= 0.0$, not small);
3. the remaining $K - q$ don't-care triplets carry $s = 0$ exactly;
4. **no rank tolerance appears anywhere**: every count comes from the masks or from bitwise $\{0, 1\}$ indicators, so exact and roundoff-level zeros need no distinction.

Equivalently: `pad` $\to$ `svd` $\to$ `unpad` $\equiv$ `svd` of the unpadded A.

Two remarks on the contract. *Symmetry:* it is stated in $q = \min(n, m)$, so any real (n, m) — tall, wide, or mixed within a batch — is supported with no feasibility condition; this matters because real counts are traced per-element data, so a runtime "transpose if wide" branch is not expressible in one batched program (Appendix A.6). *Freedom:* a valid economy SVD of A constrains its null columns only in support and orthonormality, never in direction; the completion is gauge. Demanding instead that *all* M columns of U avoid the padding would require $n \geq M$ and is both unnecessary and generally unachievable.

2 The algorithm: sketch–project

The shipped algorithm ("Method D" in the design record) rests on one observation:

Lemma 2.1 (The matvec principle)**.** *$A_{\text{pad}}x$ has bitwise-zero padded entries for every x, and $A_{\text{pad}}^\top y$ ignores the padded rows of y. Hence any method that touches A_{pad} only through products generates left vectors in* $\text{col}(A_{\text{pad}}) \perp \{e_i : i \text{ padded}\}$ *by construction.*

2.1 The permutation rule

The second ingredient makes a Householder QR respect the padding *exactly*:

Lemma 2.2 (Permutation rule)**.** *Permute the padded rows to the trailing pivot positions before a Householder QR. Then every reflector is bitwise zero on the padded rows, hence so is every data-supported column of Q; and once the real rows are exhausted the trailing submatrix is bitwise zero, so the remaining reflectors are the identity and the surplus columns of Q are* exact *pad coordinate vectors.*

Proof sketch. The reflector at step i is $v = x \mp \|x\| e_i$ with x the current column below the diagonal. Entries of v at zero rows of x vanish *except at the pivot position i*; with pads trailing, the first $\min(n, M)$ pivots are real rows, so v is zero on every padded row, and $Hy = y - 2v(v^\top y)/(v^\top v)$ preserves bitwise zeros there ($0 - 0 \cdot x = 0$). After the real rows are exhausted the trailing block consists only of (bitwise-zero) padded rows, so LAPACK's `larfg` produces $\tau = 0$, $H = I$, and Q's surplus columns are $H_1 \cdots H_n e_j = e_j$, a padded coordinate — flagged exactly by the bitwise indicator $t_j = \max_i |Q_{ij}|\,[i \text{ padded}] \in \{0, 1\}$. □

Without the permutation the failure is deterministic, not probabilistic: an in-place padded row can become a pivot, and then either the trailing block is exactly zero and $H = I$ emits $q = e_{\text{pad}}$ — a null column sitting 100% on a pad — or the trailing block is ε-level roundoff and the normalized reflector places an $O(1)$ entry at the pad coordinate. The worked example in the design record (§A.7) also shows why no *later* repair is possible: the projected core $B = Q^\top A_{\text{pad}}$ is bit-for-bit identical for a clean and a poisoned Q, so the contamination is invisible to everything downstream of the QR. Prevention at $O(NM)$ beats any repair.

2.2 The four steps

Fixed setup, once per width M: a Haar-orthonormal sketch $\Omega \in \mathbb{R}^{M\times M}$ (QR of a fixed-seed Gaussian with sign fix; $\kappa(\Omega) = 1$ avoids the mildly heavy-tailed $\sigma_{\min}$ of a square Gaussian). Ω is drawn once and cached: per-call randomness buys nothing, and a fixed sketch makes the whole algorithm deterministic. If $N < M$ (a statically wide padded shape — known at trace time), transpose, run, and swap the factors.

Step 1. Left basis. $Y = A_{\text{pad}}\Omega$ (padded rows bitwise zero for any Ω; $\text{col}(Y) = \text{col}(A_{\text{pad}})$ exactly since Ω is invertible — randomized SVD with the approximation removed, hence no oversampling parameter). Permute pads to the bottom, economy QR, un-permute $\to Q$; compute the surplus indicator t (Lemma 2.2).

Step 2. One augmented SVD. Form the statically shaped $M \times 2M$ matrix $[\,B \;\; c\,\text{diag}(t)\,]$ with $B = Q^\top A_{\text{pad}}$ and $c = 4\,\|A_{\text{pad}}\|_F$ (§2.3); take its economy SVD $W_1\Sigma_a(\cdot)$, *discarding the right factor*. Where $t_i = 1$ the diagonal block pins the surplus coordinate at $\sigma = c$ as an *exact* triplet (row i of B is bitwise zero); where $t_i = 0$ it contributes a zero column, harmless because zero columns pollute only the discarded factor. The left Gram $BB^\top + c^2\,\text{diag}(t)$ is *bitwise block-diagonal* between surplus and data coordinates, so every kept left vector — including the $\sigma = 0$ completions, which are the whole point — avoids the surplus coordinates exactly in exact arithmetic, and to $O(\varepsilon)$ in floating point.

Step 3. Classify, sort, assemble. $\sigma > c/2$ is a {pin, data} classifier with $\|A\|$-sized margins on both sides — *counting, not thresholding.* Order kept triplets descending with pins last (one `argsort` of where($\sigma > c/2$, -1, σ)); zero the (exact-arithmetic-zero) surplus coordinates of the kept columns of W_1; set $U = QW_1$ and zero s *by the pin flag*, then beyond q. The first q columns of U are bitwise zero on padded rows. Zeroing pins by flag rather than by position is what makes the contract symmetric in $\min(n, m)$: for $n < m$ the pins land inside the first m slots and positional zeroing would leak $\sigma = c$ into the spectrum (Appendix A.6).

Step 4. Rebuild V from U. $W = A_{\text{pad}}^\top U$ equals $V\Sigma$ *exactly*, with Σ decreasing, and every column bitwise supported on real columns (padded columns of A_{pad} kill those rows). Orthonormal-times-decreasing-diagonal is its own QR factorization up to column signs, recovered from $\text{sign}(\text{diag}\,R)$: the permuted QR of W (padded columns to the trailing pivots) therefore *is* V — leading columns the true right vectors, exactly, even through repeated-σ clusters; middle columns real-supported null completions; trailing columns exact pad coordinates. The whole final V, in order, tolerance-free.

Proposition 2.3 (Counting)**.** *The augmented SVD returns M triplets, of which $p_\ell = \max(0, M - n)$ are pinned; the $\min(n, M)$ survivors, sorted descending, contain the economy SVD of the real block in their first $q = \min(n, m)$ slots:* $\text{rank}(A)$ *positive values followed by exact-arithmetic zeros whose left vectors fill the real left-null space. No tolerance decides any count: m and n come from the masks, p_ℓ from the bitwise indicator t.*

2.3 The separation constant

The constant c is bounded on both sides, and both bounds were *measured* (`s1b_c_study.py` in the design record):

- **From below (fragile):** the natural-looking $c = 2\,\|A_{\text{pad}}\|_2$ puts the threshold $c/2$ exactly at $\sigma_{\max}$. The classifier compares singular values computed by two *different* factorizations (the augmented core's SVD and the norm), so whenever rounding lands the augmented $\sigma_{\max}$ one ulp above the separately computed norm, *the largest triplet is classified as a pin and silently deleted*: measured on **152 of 400** generic rank-one matrices. Treacherously, exactly-representable test data is saved by the strict inequality, so the bug survives casual testing. $c = 4\,\|A_{\text{pad}}\|_F$ gives multiplicative margins ≥ 2 on both sides: 0/400.
- **From above (a refuted worry):** one might fear a large c imposes an εc backward-error floor on the kept singular values (the augmented core has norm $\sim c$). Measured: the error stays at $\sim 10^{-16}$ even at $c = 10^{12}\,\|A\|_F$, pins present or not. The pins occupy *bitwise-zero rows* of B, so the augmented core is bitwise block-diagonal and Householder bidiagonalization keeps the blocks separated: the data block's effective backward error is $\varepsilon\,\|B\|$, not εc. The same zero-preservation that powers the algorithm firewalls its accuracy. The only genuine upper constraint is dtype overflow (relevant in `float32`).

Guard $c = 4\,\|A_{\text{pad}}\|_F + [\|A_{\text{pad}}\|_F = 0]$ so the all-zero matrix is handled; use the Frobenius norm (the spectral norm costs an SVD and buys nothing).

2.4 Cost, batching, precision

Everything is $N \times M$, $M \times 2M$, or $M \times M$; no $N \times N$ object exists. Cost $O(NM^2 + M^3)$, *independent of the pad counts* — the big matrix never sees an SVD, only QRs and matmuls (the Chan pattern, typically faster than one large SVD on GPU). All mask-dependent operations (`argsort`, gathers, diagonal indicators) are traced runtime data with static shapes: one `jit` compilation covers every mask pattern (verified: a `lax.scan` over heterogeneous masks traces its body once; the `jit` cache stays at size 1). A backend-agnostic stable pads-last permutation needs no `kind=` argument: `argsort` of the integer key $[\,i \text{ padded}\,] \cdot N + i$ has distinct keys. The bitwise-zero guarantees rest on Householder semantics (true for LAPACK, cuSOLVER, and JAX's QR lowering on all backends) and hold in `float32` unchanged — zeros are exact in every format; only orthonormality and σ-accuracy scale with precision.

2.5 Integration into the T3 sweeps

In T3Toolbox the primitive replaces every SVD that produces *kept basis columns* in the uniform sweeps: the Tucker down-orthogonalization, both TT sweeps (as a pad-safe scan step whose per-step masks ride the scan inputs), the TT up-orthogonalization, and the T3-SVD scan (unshared and shared). Three integration points are worth recording:

- **Evolving masks.** The row mask of a mid-sweep unfolding is not an input mask: bond ranks evolve by the sweep recurrences (e.g. $\ell_{j+1} = \min(\ell_j n_{D,j},\, r_{j+1})$). All evolved masks are precomputed on the host from the same recurrences the frame masks are built from, and enter the compiled program as constants.
- **A bitwise-clean chain.** Each site also *zeroes its don't-care completion columns* (they are real-supported but nonzero, and the next site's pad-safe SVD requires bitwise zero padding). The remainder side is clean automatically: s is exactly zero beyond q, so pushed remainders have bitwise-zero padded rows. The whole sweep output is in canonical clean-padding form for free.

- **Gauge.** The uniform frame used to match the ragged frame *bitwise* — an accident of both layers feeding the same matrices to the same LAPACK kernel, and impossible to preserve: at a deficient point LAPACK's completion may live in the padded slots, which is the bug. The pad-safe frame is gauge-*equivalent*: same subspaces, same singular values, same represented tensor, every gauge-invariant quantity (objectives, gradient norms, optimizer trajectories) identical — and at a deficient point even the completion *subspaces* legitimately differ, because the completion is a choice in both layers. Cross-representation comparisons must be gauge-invariant (norms, projectors $U^\top U$, dense tensors) or transport coordinates through the ambient embedding.

With the sweeps converted, the zero-padded continuation warm start keeps every tangent direction (0 lost in all six regression cases, both frame paths, shared included), and the end-to-end uniform continuation loop matches the ragged one.

A Alternatives: designed, measured, rejected

The path to Method D went through four other designs plus two "dumb" baselines. Each contributed a piece of the final algorithm; all are recorded here with what we measured. (Scripts: `dev/review_2026-08-22/repros/S1b/`.)

A.1 Method A: two-sided augmentation, and the GSVD view

Pin the pad coordinates spectrally: with one-hot mask matrices E_r, E_c and $c = 4\,\|A_{\text{pad}}\|_F$, take the SVD of

$$\begin{bmatrix} A_{\text{pad}} & c\,E_r \\ c\,E_c^\top & 0 \end{bmatrix}.$$

Each padded row/column forms an *exact* singular triplet at $\sigma = c$, separated from the data spectrum; every remaining singular vector — including the $\sigma \approx 0$ ones — is orthogonal to the c-cluster, hence supported on real coordinates. Exact, tolerance-free, and the origin of Step 2's core augmentation. Rejected as the *outer* algorithm because its cost grows with the pad count: the augmentation converts a row-pad count into extra *columns*, so with $P_r \sim N/2$ the QR costs $O(N^3)$ against $O(NM^2)$ unaugmented. Method D applies the same augmentation at the $M \times 2M$ core, where it costs $O(M^3)$.

The GSVD equivalence (checked numerically, `s1b_gsvd.py`): the augmentation is the generalized SVD of the pair $(A_{\text{pad}}^\top, B)$ with $B^\top B = P_{\text{real}}$ [1, 2]. Because $P_{\text{real}} A_{\text{pad}} = A_{\text{pad}}$ the pair *commutes*, and the generalized singular pairs sort the space into the data spectrum (untouched), the real complement (the completion), and the common null space (the padding); the augmented matrix $[A_{\text{pad}} \;\; \varepsilon B]$ has Gram $A_{\text{pad}} A_{\text{pad}}^\top + \varepsilon^2 P_{\text{real}}$ — the same commuting pair folded into one operator — and LAPACK computes the GSVD as a QR-plus-CS decomposition of the stacked matrix, which *is* this augmentation. In exact arithmetic ε affects only the ordering, so there is no threshold. A thin variant for suffix-padded sites ($B =$ the first n masked coordinate vectors) was verified over 400 random cases at $O(Nn^2)$; Method D subsumed it by handling interior pads with the same machinery.

A.2 Method B: post-hoc masked completion

Run the plain SVD; keep the $\sigma > \tau$ triplets (their left vectors are automatically clean, $u = A_{\text{pad}} v/\sigma$); *replace* the rest by explicitly constructed real-supported vectors: a masked random

block, orthogonalized against the kept columns (what TeNPy does by hand). Cost $O(NM^2)$, independent of the pad count; exact for the kept vectors. Rejected because it reintroduces a rank tolerance τ and a per-element number of replacements — under `jit` that is `where` / `take_along_axis` bookkeeping for a result Method D produces with no threshold at all. Method D's Step 4 is exactly this completion realized through the permutation rule instead of a threshold.

A.3 Methods C, C′: masked noise, and Rayleigh–Ritz refinement

Add a tiny perturbation ηG supported on the real rows/columns and run the plain SVD: every real direction gets $\sigma \gtrsim \eta > 0$ and outranks the padding, and the lifted vectors are real-supported because G is. The one real advantage: the matrix stays $N \times M$ (no extra columns). The costs: the noisy Gram carries first-order cross terms $A_{\text{pad}}G^\top + GA_{\text{pad}}^\top$ — the kept singular vectors rotate at $O(\eta/\text{gap})$ where the augmentation perturbs at second order (and not at all for the commuting B) — values $\sigma \lesssim \eta$ are erased (η is an implicit rank threshold), the balanced error is $\sqrt{\varepsilon}\,\|A\|$ (the Gram-route half-precision loss, and for the same reason), and random noise destroys determinism (reproducibility, cache keys, cross-representation tests); a fixed dither fixes determinism and is then exactly "Method A with the candidate block added into A_{pad} instead of appended". A Rayleigh–Ritz pass on the clean matrix (C′) improves the balanced error to $\varepsilon^{2/3}\,\|A\|$ but cannot remove pad mass (a core-level rotation only redistributes it). Kept in the record for one genuine niche: inside a differentiable pipeline the lifting also regularizes the SVD derivative at repeated σ, which none of the exact methods do.

A.4 Round-tripping through the ragged layer

Convert to ragged, run the per-element sweep (whose completions are real by construction), pack back. Exact — and not expressible under `jit`: the conversion produces data-dependent shapes, the sweep is $C \cdot d$ small host-side kernels, and the compile-once loop breaks at every frame rebuild. Viable only eagerly, where the uniform layer is already the slower path, so it never competes where the problem actually lives.

A.5 Shrinking the masks to the numerical rank

Treat the deficient point as a lower-rank point. Rejected outright: rank continuation exists precisely to *grow into* the padded directions; shrinking the masks removes the escape directions the warm start is built to provide.

A.6 The symmetric contract needs no runtime transpose

The packet's original contract required $n \geq m$ (real rows $\geq$ real columns). Since real counts are traced per-element data, dispatching on them would need `lax.cond` — which under batching executes *both* branches — and per-element transposition inside one batched linalg call is not expressible at all. Measured resolution (`s1b_sym_variant.py`): the restriction is a contract statement, not an algorithmic one. Zeroing the pinned σ's *by flag* instead of by position (Step 3) is the only change needed; the same pipeline then serves any real (n, m) — verified over 600 random cases spanning tall, wide, and statically-wide shapes with interior pads, rank deficiency, and zero real rows and columns. The only transpose left is the static one, a Python branch on concrete padded shapes.

A.7 The core is blind: why prevention beats repair

Take the 5×3 walkthrough of the design record: rank-one data, interior pad rows, one numerically zero *real* row, entries chosen so the QR is exact in floating point. With the permutation, Q's null column lands on the zero *real* row (allowed — the requirement is one-sided); without it, $H = I$ emits an exact pad coordinate — total contamination, deterministically. But in *both* cases the projected core $B = Q^\top A_{\text{pad}}$ is bit-for-bit

$$B = \begin{pmatrix} 5 & 10 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix} :$$

a zero row of B says only "this column of Q is orthogonal to $\text{col}(A_{\text{pad}})$" — it cannot distinguish a clean null column from a poisoned one. The contamination lives entirely in Q, upstream of anything the core sees, so no core-level repair is possible; the permutation rule at $O(NM)$ is the only place the information exists.

A.8 Summary of measurements

claim	measurement	script
warm start loses directions (pre-fix)	60/60 trials, cascades through the sweep	`s1b_cases.py`
T3-SVD tensor immune	60/60 exact	`s1b_t3svd_sweep.py`
GSVD $\equiv$ augmentation	overlap 1.000, $\sigma > 0$ exact	`s1b_gsvd.py`
thin candidates at suffix sites	400/400, $O(Nn^2)$	`s1b_tucker_aug.py`
symmetric contract, no transpose	600/600 tall/wide/static-wide	`s1b_sym_variant.py`
$c = 2\|A\|_2$ deletes top triplet	152/400 generic rank-1	`s1b_c_study.py`
εc floor refuted	σ error 10^{-16} at $c = 10^{12}\|A\|_F$	`s1b_c_study.py`
one compile across mask patterns	`jit` cache size 1 (scan + direct)	`test_dispatch.py`
post-fix: 0 lost directions	all 6 cases, both frame paths, shared incl.	`TestPadSafeFrame`

References

[1] C. F. Van Loan, *Generalizing the singular value decomposition*, SIAM J. Numer. Anal. 13 (1976) 76–83.

[2] C. C. Paige and M. A. Saunders, *Towards a generalized singular value decomposition*, SIAM J. Numer. Anal. 18 (1981) 398–405.

[3] G. Ceruti, J. Kusch, and C. Lubich, *A rank-adaptive robust integrator for dynamical low-rank approximation*, BIT 62 (2022) 1149–1174. arXiv:2104.05247.

[4] C. Vermeylen, B. Vandereycken, and P.-A. Absil, *Rank-adaptive Riemannian optimization for tensor completion in the tensor-train format*, 2024. arXiv:2402.12182.

[5] C. Hubig, I. P. McCulloch, U. Schollwöck, and F. A. Wolf, *Strictly single-site DMRG algorithm with subspace expansion*, Phys. Rev. B 91 (2015) 155115. arXiv:1501.05504.

[6] T3Toolbox documentation: `docs/uniform_equivalence_contract.md`, `docs/contributor/uniform_svd_prefix_orthogonalization.md`, `docs/rank_continuation.md`. Code: `t3toolbox/backend/linalg.py` (`pad_safe_svd`); tests: `tests/backend/test_linalg.py`, `TestPadSafeFrame` in `tests/test_uniform_frame_variations_format.py`.